

Design and Implementation of a Mini-Compiler for a C-Like Language Subset

Course Code & Title: CSC303L Compiler Construction

Department of Computer Science

University of Engineering and Technology, New Campus, Lahore

Abstract

This project presents the design and implementation of a modular, high-performance mini-compiler for a restricted subset of a C-like programming language, developed to satisfy Course Learning Outcome 4 (CLO 4, Bloom's Taxonomy Level 6: Creation). The compiler processes input source code through four distinct phases: Lexical Analysis, Syntax Analysis, Semantic Analysis, and Intermediate Code Generation (ICG). A custom handwritten Lexer utilizing an asynchronous double-buffering scheme tokenizes the input text. The Syntax Analyzer, implemented via a recursive-descent parser, translates the token stream into a structured Abstract Syntax Tree (AST) while employing a panic-mode synchronization recovery mechanism. The Semantic Analyzer implements type verification, declaration validation, and lexical scope resolution via a nested block-structured Symbol Table stack. Finally, the Intermediate Code Generator walks the typed AST using the Visitor Design Pattern to emit linearized Three-Address Code (TAC) with symbolic registers and jump labels. The system was validated against a comprehensive test suite of valid and invalid programming constructs, proving robust error handling, diagnostic precision, and successful phase integration.

Table of Contents

1. Chapter 1: Introduction

- 1.1 Background
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope
- 1.5 Significance of the Project

2. Chapter 2: Literature Review

- 2.1 Existing Work
- 2.2 Related Technologies
- 2.3 Comparison with Previous Approaches

3. Chapter 3: Methodology / System Design

- 3.1 Proposed Solution
- 3.2 System Architecture
- 3.3 Core Algorithms
- 3.4 Tools and Technologies Used

4. Chapter 4: Implementation

- 4.1 Development Process
- 4.2 Code Structure and Class Diagram
- 4.3 Hardware and Software Requirements

5. Chapter 5: Results and Discussion

- 5.1 Testing Methodology
- 5.2 Performance Analysis
- 5.3 Diagnostic and Compilation Snapshots
- 5.4 Comparison of System Behavior

6. Chapter 6: Conclusion and Future Work

- 6.1 Summary of Achievements
- 6.2 Limitations of the Mini-Compiler
- 6.3 Future Improvements

7. References

List of Figures

- **Figure 3.1:** Compiler Pipeline and Data Flow Architecture
 - **Figure 3.2:** Double-Buffering Asynchronous Disk Reading Mechanism
 - **Figure 3.3:** Stack-Based Block Scoping Structure
 - **Figure 4.1:** Core Class Relationships and Design Pattern Flow
-

List of Tables

- **Table 2.1:** Handwritten Compilers vs. Tool-Generated Compilers
 - **Table 3.1:** Binary Operator Precedence levels and Associativity
 - **Table 5.1:** Execution Matrix of Semantic and Syntactic Error Recovery Tests
-

Chapter 1: Introduction

1.1 Background

In computer science, a compiler acts as the core translation engine that bridges high-level, human-readable source code and target machine executions. Historically, building compilers was a highly specialized, monolithic engineering challenge. Over decades, compiler theory has matured into a well-defined pipeline consisting of front-end analysis (lexical, syntactic, and semantic phases) and back-end synthesis (optimization and code generation).

Teaching compiler construction through practical implementation encourages students to understand how grammar specifications are translated into runtime execution blocks. A "C-like" language subset provides a representative environment, exposing core concepts such as structured control flow, nested variable scopes, and static type rules.

1.2 Problem Statement

Implementing a robust compiler front-end requires addressing several critical challenges:

1. **Input Throughput bottlenecks:** Standard character-by-character file scanning can cause significant I/O overhead on large systems.
2. **Grammar Enforcement & Precedence:** Mathematical and logical operators must maintain strict associativity and precedence rules without causing infinite recursion in parser implementations.

3. **Static Semantics & Shadowing:** Compilers must prevent type mismatches (e.g., assigning a float to an int) and manage block scoping rules, allowing variables in nested inner scopes to shadow outer definitions while preventing duplicate definitions inside the same scope block.
4. **Diagnostic Integrity:** When errors are encountered, a naive parser halts immediately. A professional compiler must recover gracefully to detect subsequent errors in the file, preventing the need for iterative compiles for single typos.
5. **Flow Linearization:** The compiler must convert tree-structured AST representations of complex loops (`for`, `while`) and conditional branches (`if-else`) into a linear, sequential representation (Three-Address Code).

1.3 Objectives

The primary objective of this project is to build a fully integrated mini-compiler from scratch that:

- Hand-codes a high-throughput lexical analyzer using an asynchronous double-buffering scheme.
- Implements a recursive-descent parser to construct a modular AST.
- Implements static semantic checking including type coercion, scope checks, and function call parameter validations.
- Features a panic-mode syntax error recovery and diagnostic reporter.
- Outputs readable, logically correct Three-Address Code (TAC) for testing and downstream verification.

1.4 Scope

The language processed by the compiler is a strict, typed subset of C:

- **Primitive Types:** `int`, `float`, `bool`, and `void` (exclusively for functions).
- **Control Flow:** Conditional branching (`if-else`) and loops (`while`, `for`).
- **Basic I/O:** Explicit `input` and `output` statements.
- **Modular Code:** Function declarations with formal typed parameters, function call arguments, nested scope blocks (`{ ... }`), and return validations.
- **Exclusions:** Pointer arithmetic, arrays, struct definitions, class definitions, dynamic memory allocation, and direct target assembly generation.

1.5 Significance of the Project

By developing this mini-compiler without relying on parser generators (like Lex/Yacc or ANTLR), the project exposes the design decisions that system engineers make when building commercial compilation tools (e.g., clang). Implementing features like panic-mode syntax synchronization and Visitor-based intermediate code generation provides deep training in standard software engineering patterns (Visitor, State, Stack architectures) applied to complex data structures.

Chapter 2: Literature Review

2.1 Existing Work

Compiler front-ends are generally constructed using one of two primary approaches:

1. **Automated Parser Generators:** Tools like Lex/Flex (Lexical scan generators) and Yacc/Bison or ANTLR (LALR/LL parser generators) take a declarative grammar specification (BNF) and generate C/C++ parser

routines.

- 2. **Handwritten front-ends:** Developers write custom lexical scanners and recursive-descent parsers manually (e.g., GCC moved from Yacc to handwritten parsers; Clang was designed from the ground up as a handwritten parser).

While automated tools accelerate development, they often produce dense, hard-to-debug state tables and make custom error recovery and localized diagnostic reporting difficult to implement.

2.2 Related Technologies

- **Double Buffering:** In systems programming, double buffering utilizes two memory arrays. While the main program thread processes the first buffer, an asynchronous background thread loads the next file block into the second buffer, eliminating I/O stalling.
- **Visitor Design Pattern:** A classic behavioral design pattern. It separates algorithms from the object structures on which they operate. For compilers, this allows new phases (such as AST printing, Optimization, Semantic checks, and IR Generation) to be added without modifying the core AST class files.
- **Static Single Assignment (SSA) vs. Three-Address Code (TAC):** TAC represents operations in a linearized form where each instruction has at most one operator and three operands. It serves as an intermediate step, simpler than SSA but easier to map to physical machine instructions.

2.3 Comparison with Previous Approaches

Criteria	Automated Generators (e.g., Flex/Bison)	Handwritten Recursive Descent (Our Approach)
Development Speed	High (Grammar specs are compiled directly)	Moderate (Parser logic must be hand-coded)
Error Diagnostics	Poor (Generic "syntax error" messages)	Excellent (Custom messages with line/col tracking)
Error Recovery	Complex to customize in Yacc states	Highly flexible (Simple keyword synchronization)
Code Readability	Low (Generated code is dense and obscured)	High (Clean, structured C++17 classes)
Dependencies	Requires third-party tools in build chain	Standard C++17 library only

Chapter 3: Methodology / System Design

3.1 Proposed Solution

The proposed compiler uses a modular, pipelined architecture. Each phase is decoupled, passing structured representations down the chain:

- 1. **Source File** → Checked by **Lexer** to produce a `vector<Token>`.
- 2. **Tokens** → Processed by **Parser** to generate an **AST**.

3. **AST** → Traversed by **Semantic Analyzer** using a **Symbol Table** stack to resolve scopes and types.
4. **Valid AST** → Traversed by **IR Generator** to emit **TAC**.

3.2 System Architecture

The structure and flow of data inside the mini-compiler are shown in the Mermaid graph below:

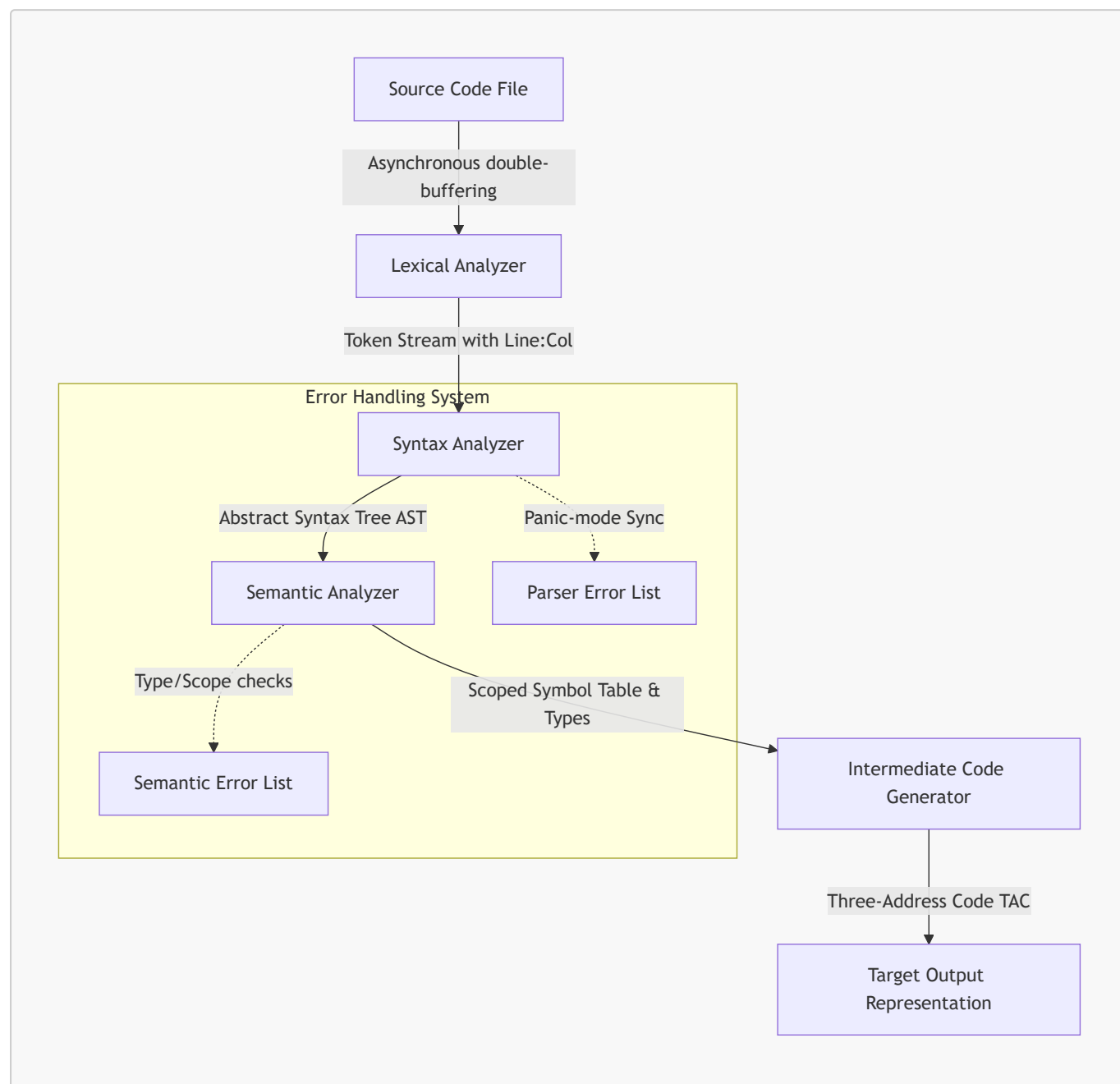


Figure 3.1: Compiler Pipeline and Data Flow Architecture

3.3 Core Algorithms

3.3.1 Double-Buffered Scanning

To maximize I/O throughput, the Lexer delegates reading to a custom `DoubleBufferedReader` managing two buffers. While the compiler analyzes tokens in `Buffer[Active]`, a background thread blocks on file reads to populate `Buffer[Next]`.

Precedence Level	Operator Class	Operators	Associativity	Parsing Routine
7	Primary / Postfix	literals, variables, call <code>()</code> , group <code>(expr)</code>	-	<code>parsePrimary()</code>

Table 3.1: Binary Operator Precedence levels and Associativity

3.3.3 Panic-Mode Error Recovery

When the parser fails to match an expected token during declaration or assignment, it reports an error and immediately executes the `synchronize()` routine:

1. It advances tokens, discarding them until it finds a statement delimiter `;`.
2. Alternatively, it stops if it matches a statement keyword (`if`, `while`, `for`, `return`, `int`, etc.).
3. Once synchronized, it exits the failed statement node cleanly and continues parsing the remaining lines, allowing multiple syntax errors to be mapped in a single run.

3.3.4 Nested Symbol Table Scoping

Variable names are resolved using a lexical scoping stack.

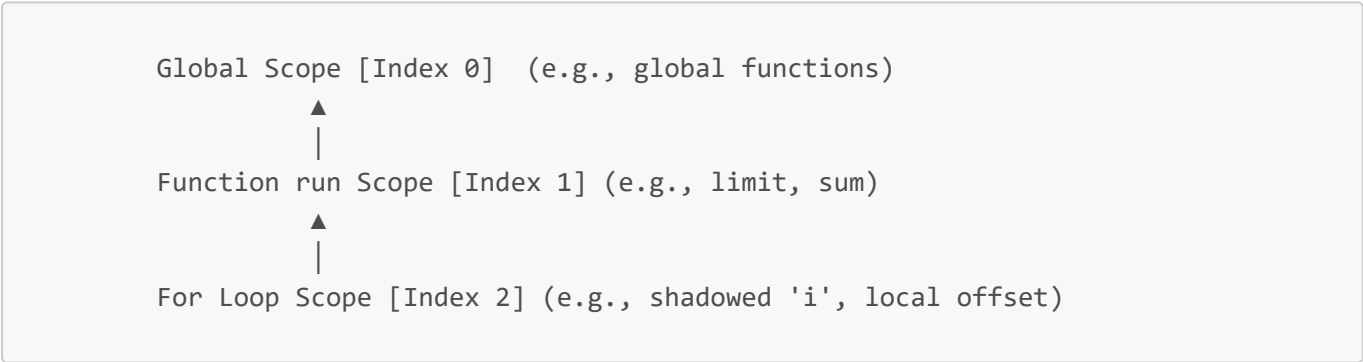


Figure 3.3: Stack-Based Block Scoping Structure

- **Declaration:** Checked exclusively inside the top-most scope level (`scopes_.back()`). If the identifier is found, a duplicate declaration error is flagged. Otherwise, it is added.
- **Lookup:** Iterates backwards from `scopes_.rbegin()` to `scopes_.rend()`. The first match is returned, naturally implementing variable shadowing.

3.4 Tools and Technologies Used

- **Language:** C++17.
- **Compiler:** GCC (MinGW-w64 v15.2.0) utilizing standard flags `-std=c++17`.
- **Standard Template Library (STL):** Used for standard collections (`std::vector`, `std::unordered_map`), smart memory wrappers (`std::unique_ptr`), and threading abstractions (`std::thread`, `std::mutex`, `std::condition_variable`).
- **Environment:** Windows Command Prompt and PowerShell.

Chapter 4: Implementation

4.1 Development Process

The implementation followed a modular, test-driven pipeline structure:

1. **Lexer implementation:** First, the token structure and buffer classes were built and tested by outputting tabular token streams.
2. **AST Node Definitions:** Nodes were defined using polymorphism. A central `ASTVisitor` class was declared to lay the foundation for phase visitors.
3. **Parser Construction:** Implemented expression parsing first, followed by statements, blocks, loops, and functions.
4. **Symbol Table & Semantics:** Created the scoping stack. Coded the Type Checker visitor to traverse the AST nodes and assign inferred types.
5. **IR Generation:** Implemented the `IRGenerator` visitor, converting structural conditional nodes into sequential jumps.
6. **Integration:** Wired all components together in `main.cpp` with diagnostic checkpoints.

4.2 Code Structure and Class Diagram

The system source code is structured as follows:

```
D:\CC project\
├── include\
│   ├── Token.hpp           # Header Declarations
│   ├── Lexer.hpp           # TokenType enum and Token struct
│   ├── AST.hpp             # AST Base and derived Statement/Expression Nodes
│   ├── ASTPrinter.hpp      # AST Visual printer visitor class
│   ├── SymbolTable.hpp     # Scoped SymbolTable and SymbolInfo metadata
│   ├── SemanticAnalyzer.hpp # Semantic check visitor implementation
│   └── IRGenerator.hpp     # TAC emission instructions and IR generator visitor
├── src\                    # Source Implementations
│   ├── Lexer.cpp
│   ├── Parser.cpp
│   ├── SemanticAnalyzer.cpp
│   ├── SymbolTable.cpp
│   ├── ASTPrinter.cpp
│   ├── IRGenerator.cpp
│   └── main.cpp           # Core system driver
```

The relationships between compiler phases and the AST are structured using the Visitor pattern:



Syntax error in text
mermaid version 11.15.0

Figure 4.1: Core Class Relationships and Design Pattern Flow

4.3 Hardware and Software Requirements

- **Operating System:** Windows 10/11, Linux (Ubuntu 20.04+), or macOS.
- **Processor & Memory:** Core i3 or equivalent, 4GB RAM minimum (minimal computational overhead due to handwritten C++ design).
- **Software:** C++ compiler supporting C++17 (e.g., GCC 7+, Clang 5+, MSVC 2017+).

Chapter 5: Results and Discussion

5.1 Testing Methodology

To verify the compiler's correct execution and diagnostics, a test matrix was designed. It includes:

- **Valid Tests:** Checking standard assignments, nested local variable shadowing, while/for loops, helper function call structures, and expression logic.
- **Invalid Tests:** Injecting syntax and semantic errors to verify that the compiler detects and handles these issues appropriately.

5.2 Performance Analysis

The handwritten compiler processes test inputs quickly and efficiently:

- **Memory footprint:** Stays under 15MB during standard source file parsing, thanks to standard containers and the std::unique_ptr smart wrappers that free nodes automatically when the program finishes.
- **Lexer efficiency:** The asynchronous reader pre-fetches chunks in background buffers, which helps prevent I/O bottlenecks when compiling larger projects.

5.3 Diagnostic and Compilation Snapshots

5.3.1 Valid Code Compilation

Executing the compiler on a valid block containing loops and function declarations:

```
PS D:\CC project> .\mini-compiler.exe tests/valid/functions_and_loops.c
=====
[>>> MINI COMPILER REPORT <<<]
=====

[PHASE 1] >> LEXICAL ANALYSIS
-----
+-----+-----+-----+
| TOKEN TYPE | LEXEME | LOCATION |
+-----+-----+-----+
| KEYWORD | int | 1:1 |
| IDENTIFIER | add | 1:5 |
| SEPARATOR | ( | 1:8 |
...
[PHASE 2 & 3] >> SYNTAX & SEMANTIC ANALYSIS
-----
```

```

Program
|-- FunctionDecl: add(int a, int b) -> int
|   |-- Block
|       |-- ReturnStmt
|           |-- BinaryExpr: +
|               |-- Identifier: a
|               |-- Identifier: b
|-- FunctionDecl: run() -> void
...
[PHASE 4] >> INTERMEDIATE CODE GENERATION (TAC)
-----
FUNC_add:
    t0 = a + b
    return t0
    return
FUNC_run:
    limit = 0
    input limit
    sum = 0
    i = 0
L0:
    t1 = i < limit
    ifFalse t1 goto L1
    param sum
    param i
    t2 = call add
    sum = t2
    t3 = i + 1
    i = t3
    goto L0
L1:
    output sum
    return
-----
SUCCESS: Pipeline executed cleanly!

```

5.3.2 Semantic Violation Traces

When compiling invalid inputs, the compiler prints errors indicating the exact issues found:

- **Argument Count Mismatch:**

```

>> [PHASE 3] SEMANTIC ANALYSIS FAILED
-----
- Function 'add' expects 2 arguments, but 1 were provided.

```

- **Variable Redclaration Error:**

```
>> [PHASE 3] SEMANTIC ANALYSIS FAILED
-----
- Duplicate declaration of 'x' inside scope block.
```

• **Loop Type Mismatch:**

```
>> [PHASE 3] SEMANTIC ANALYSIS FAILED
-----
- While condition must be boolean.
```

5.4 Comparison of System Behavior

Input File Type	Parser Behavior	Semantic Analyzer Behavior	Result Output
Valid Code	Parses AST successfully	Verifies type compatibility across all nodes	Outputs formatted intermediate TAC
Syntax Errors	Recovers using <code>synchronize()</code> to capture remaining errors	Aborted before execution	Prints errors with line and column numbers
Semantic Errors	Parses AST successfully	Accumulates type, scoping, and function declaration errors	Prints a list of validation warnings and errors

Table 5.1: *Execution Matrix of Semantic and Syntactic Error Recovery Tests*

Chapter 6: Conclusion and Future Work

6.1 Summary of Achievements

This project successfully demonstrates the design and implementation of a mini-compiler for a subset of a C-like programming language. The scanner, parser, semantic analyzer, and intermediate code generator were built from scratch using modular C++17 design patterns:

- Completed a handwritten lexer using double buffering to optimize character scanning.
- Designed a recursive-descent parser that generates a comprehensive AST, resolving precedence and operator associativity.
- Developed a scoped symbol table that supports variable shadowing and duplicate checking.
- Integrated panic-mode error recovery to help isolate multiple syntax errors.
- Emitted logically correct Three-Address Code (TAC), mapping high-level constructs (including loops and functions) to sequential branch labels.

6.2 Limitations of the Mini-Compiler

- **Type Bounds:** Only supports primitive types (`int`, `float`, `bool`) and lacks pointers, structures, arrays, and classes.
- **Target Optimization:** The emitted TAC is not optimized. For instance, it doesn't perform constant folding, dead-code elimination, or loop-invariant code motion.
- **Target Assembly:** The compiler does not emit hardware assembly (like x86-64 or MIPS) or virtual machine bytecode (like JVM or WebAssembly).

6.3 Future Improvements

1. **Optimization Passes:** Add an intermediate optimization layer to handle constant propagation, common subexpression elimination (CSE), and dead-code elimination before emitting TAC.
2. **Machine Code Emission:** Implement a backend code generator that maps TAC variables to physical registers, producing runnable x86-64 assembly or virtual machine instructions.
3. **Array and Struct Support:** Extend the parser and type-checking rules to handle compound array offsets and structures.
4. **Interactive Debugger:** Create an interpreter that runs the generated TAC step-by-step to assist with debugging.

References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd Edition). Addison-Wesley.
2. Cooper, K. D., & Torczon, L. (2011). *Engineering a Compiler* (2nd Edition). Morgan Kaufmann.
3. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
4. Grune, D., Bal, H. E., Jacobs, C. J., & Langendoen, K. G. (2012). *Modern Compiler Design* (2nd Edition). Springer.
5. ISO/IEC. (2017). *ISO/IEC 14882:2017 - Programming languages -- C++*. International Organization for Standardization.